arXiv:2402.06082v1 [cs.LG] 8 Feb 2024

SubGen: Token Generation in Sublinear Time and Memory

Amir Zandieh*
Independent Researcher
amir.zed512@gmail.com

Insu Han*
Yale University
insu.han@yale.edu

Vahab Mirrokni
Google Research
mirrokni@google.com

Amin Karbasi
Yale University, Google Research
amin.karbasi@yale.edu

Abstract

Despite the significant success of large language models (LLMs), their extensive memory requirements pose challenges for deploying them in long-context token generation. The substantial memory footprint of LLM decoders arises from the necessity to store all previous tokens in the attention module, a requirement imposed by key-value (KV) caching. In this work, our focus is on developing an efficient compression technique for the KV cache. Empirical evidence indicates a significant clustering tendency within key embeddings in the attention module. Building on this key insight, we have devised a novel caching method with sublinear complexity, employing online clustering on key tokens and online ℓ_2 sampling on values. The result is a provably accurate and efficient attention decoding algorithm, termed SubGen. Not only does this algorithm ensure a sublinear memory footprint and sublinear time complexity, but we also establish a tight error bound for our approach. Empirical evaluations on long-context question-answering tasks demonstrate that SubGen significantly outperforms existing and state-of-the-art KV cache compression methods in terms of performance and efficiency.

1 Introduction

Large Language Models (LLMs) [1, 23] play a crucial role in various natural language processing applications, including dialog systems [22, 5], coding assistance [4, 18], and image/video generations from text [17, 11]. All of these models rely on the transformer architecture, with the attention mechanism serving as the key component.

To fully harness the capabilities of LLMs, they must demonstrate both efficiency and accuracy in generating long sequences. In practical applications, deploying LLMs to generate tokens in an autoregressive manner involves a sequential decoding process, where attention is dynamically applied to each newly generated token. This process effectively constructs the output sequence in a streaming manner, one token at a time. Therefore, as the sequence grows, the model has to produce contextually relevant and coherent content.

A common method for autoregressive attention decoding involves the use of key-value (KV) caching, where key and value pairs from *all* preceding tokens are cached and reused to prevent redundant

*Equal Contributions

computations. However, this approach faces memory constraints, particularly when handling long sequences. In particular, the memory requirements and runtime for generating each new token increase linearly with context size, posing a significant challenge for efficient processing of extensive sequences. This linear scaling directly impedes practical applicability in real-world scenarios, such as chat systems, where large contexts are often encountered.

In this work, we delve into the primary computational and memory bottleneck of token generation. We propose SubGen, a novel approach designed to significantly reduce the memory and runtime complexity of token generation, moving from conventional linear growth to sublinear scale. To summarize, our goal is to answer the following question:

Can we approximate the attention output in decoding phase
in sublinear space/time complexity in context length?

1.1 Related Work

Recent studies have underscored the need for efficient token generation, particularly with the rise of long-range context datasets. Several recent works have developed efficient strategies for compressing the KV cache. Zhang et al. [28] proposed a greedy-type eviction algorithm that dynamically keeps at most $k \ll n$ token embeddings based on the accumulated attention scores where they refer to the Heavy Hitter Oracle (H2O). Liu et al. [14] empirically observed that tokens with initially high attention scores tend to stay high during the future generation process. Motivated by this observation, the authors proposed a strategy that only keeps the most recent and pivotal tokens whose attention scores are higher than a threshold. Ge et al. [9] proposed an adaptive method of KV cache compression which identifies the intrinsic structures of attention heads and uses them to determine the optimal compression policy. Xiao et al. [26] observed that a simple eviction mechanism that keeps only the first few and last few tokens does not degrade much the decoding quality. They additionally proposed a fine-tuning method to solve performance degradation from their method. Liu et al. [15] developed an algorithm that reduces the generation latency by exploiting contextual sparsity. In addition to algorithmic acceleration, there has also been a line of work optimizing hardware resource configurations [20, 12]. However, to the best of our knowledge, none of these works have achieved an efficient method for KV cache with fully sublinear-time memory space.

On the lower bound side, achieving subquadratic amortized runtime for producing output embeddings for n tokens in the worst-case instances is likely impossible without making assumptions about the input tokens [2, 19]. Therefore, to achieve fast runtime, it is necessary to rely on certain assumptions about the input tokens.

1.2 Streaming Attention Problem

Deployment of LLMs involves performing attention decoding in a streaming fashion. More precisely, the stream of tokens is represented as a sequence of vector triplets $(\boldsymbol{q}_1, \boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{q}_2, \boldsymbol{k}_2, \boldsymbol{v}_2), \ldots (\boldsymbol{q}_n, \boldsymbol{k}_n, \boldsymbol{v}_n)$, where $\boldsymbol{q}_i, \boldsymbol{k}_i, \boldsymbol{v}_i \in \mathbb{R}^d$ are queries, keys, and values of the attention mechanism and n is the total

number of tokens in the stream so far either in prompt or generation. [1] The objective of streaming attention decoding is to compute the following:

$$\mathrm{Attn}(\boldsymbol{q}_n, \boldsymbol{K}_n, \boldsymbol{V}_n) = \texttt{softmax}(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)^\top \cdot \boldsymbol{V}_n, \tag{1}$$

where $\boldsymbol{K}_n, \boldsymbol{V}_n \in \mathbb{R}^{n \times d}$ are matrices defined by stacking the keys and values in their respective rows:

$$\boldsymbol{K}_n := \begin{bmatrix} \boldsymbol{k}_1^\top \\ \boldsymbol{k}_2^\top \\ \vdots \\ \boldsymbol{k}_n^\top \end{bmatrix}, \quad \boldsymbol{V}_n := \begin{bmatrix} \boldsymbol{v}_1^\top \\ \boldsymbol{v}_2^\top \\ \vdots \\ \boldsymbol{v}_n^\top \end{bmatrix}. \tag{2}$$

The output $\mathrm{Attn}(\boldsymbol{q}_n, \boldsymbol{K}_n, \boldsymbol{V}_n)$ is then used for predicting the next token and its token embedding is applied to a transformer model and a new stream pair $(\boldsymbol{q}_{n+1}, \boldsymbol{k}_{n+1}, \boldsymbol{v}_{n+1})$ is generated. However, storing these values and keys requires $O(nd)$ memory, posing a significant space complexity challenge for long-context models with large n.

1.3 Overview of Contributions

We propose SubGen, an efficient method that accurately approximates the attention decoder's output in Eq. (1) while retaining only a small (sublinear) subset of keys and values in the cache. In particular, SubGen computes an estimator $\boldsymbol{z}_n$ for $\mathrm{Attn}(\boldsymbol{q}_n, \boldsymbol{K}_n, \boldsymbol{V}_n)$ in sublinear time and memory such that the error is bounded as follows:

$$\|\boldsymbol{z}_n - \mathrm{Attn}(\boldsymbol{q}_n, \boldsymbol{K}_n, \boldsymbol{V}_n)\|_2 \leq \varepsilon \, \|\texttt{softmax}(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)\|_2 \, \|\boldsymbol{V}_n\|_{op}. \tag{3}$$

This error bound is in line with the spectral errors studied in previous works [27, 10].

We begin by observing that $\mathrm{Attn}(\boldsymbol{q}_n, \boldsymbol{K}_n, \boldsymbol{V}_n)$ in Eq. (1) is the product of the softmax vector $\texttt{softmax}(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)$ and value matrix $\boldsymbol{V}_n$. This matrix-vector product can be approximated by subsampling only $O(\varepsilon^{-2} d \log n)$ key-value pairs according to the vector and matrix according to the squared norms of value tokens. This can be implemented in a streaming setting using some variants of reservoir sampling.

The other major computational challenge is computing the partition function in the denominator of the softmax function, i.e., $\sum_{i \in [n]} \exp(\langle \boldsymbol{k}_i, \boldsymbol{q}_n \rangle)$. To solve this, we construct a data structure that can be stored in sublinear memory and efficiently approximate $\sum_{i \in [n]} \exp(\langle \boldsymbol{k}_i, \boldsymbol{q}_n \rangle)$ up to $1 \pm \varepsilon$ factor for any query $\boldsymbol{q}_n$. Our method assumes that the key tokens can be covered by a sublinear number of bounded diameter clusters. This assumption is indeed weaker than the one made in [10], which in the decoding setting translates to having key tokens belong to only one cluster with a bounded diameter, while our approach allows for any sublinear number of clusters. So, if the keys are composed of bounded diameter clusters then we only need a small number of uniformly sampled keys from each cluster to approximate the softmax normalizer efficiently and accurately. The central task is to find these clusters in a streaming setting, and we achieve this using an algorithm that is inspired by the streaming k-center algorithm of [3].

[1] We denote vectors with lowercase boldface letters, e.g., $\boldsymbol{v}$, matrices with uppercase boldface letters, e.g., $\boldsymbol{M}$, and sets with calligraphy uppercase letters, e.g., $\mathcal{S}$. The operator norm of a matrix is denoted as $\|\cdot\|_{op}$.

In Theorem 1 and Corollary 1 we demonstrate that if the keys can be clustered into some sublinear number $m = n^{1-\Omega(1)}$ of clusters with some bounded diameters, then SubGen operates with sublinear $O\left(\varepsilon^{-2}md\right) = O\left(\varepsilon^{-2}dn^{1-\Omega(1)}\right)$ memory and runtime and its output satisfies the approximation guarantee in Eq. (3).

In Section 3, we empirically compare SubGen to other KV cache compression methods including the attention-score-based algorithm of [28] and the deterministic eviction policy from [26]. Our results confirm that SubGen outperforms these methods, particularly in question-answering tasks with various sequence lengths.

2 Sublinear Time and Memory Algorithm

Our goal is to approximate the attention output in Eq. (1) with a space complexity that is sublinear in context length n. To achieve this objective, we aim to design the following data structure (DS) for efficiently approximating the streaming attention mechanism:

2.1 Streaming Attention Data Structure

For every positive integer n and every stream of token triplets $(\boldsymbol{q}_1, \boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{q}_2, \boldsymbol{k}_2, \boldsymbol{v}_2), \ldots (\boldsymbol{q}_n, \boldsymbol{k}_n, \boldsymbol{v}_n)$ where $\boldsymbol{q}_i, \boldsymbol{k}_i, \boldsymbol{v}_i \in \mathbb{R}^d$, we aim to construct an efficient DS with the following properties:

- The required memory space is sublinear in n, i.e., $o(n)$.
- Upon the arrival of a new triplet $(\boldsymbol{q}_{n+1}, \boldsymbol{k}_{n+1}, \boldsymbol{v}_{n+1})$ in the stream, the time complexity to update is sublinear in n, i.e., $o(n)$.
- Given such data structure, there exists an algorithm that outputs an estimator $\boldsymbol{z}_n \in \mathbb{R}^d$ in sublinear time $o(n)$ such that:

$$\left\| \boldsymbol{z}_n - \texttt{softmax}(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)^\top \cdot \boldsymbol{V}_n \right\|_2 \leq \varepsilon \left\| \texttt{softmax}(\boldsymbol{K}_n \cdot \boldsymbol{q}_n) \right\|_2 \left\| \boldsymbol{V}_n \right\|_{op}. \tag{4}$$

In the rest of this section, our focus is on developing an algorithm to satisfy the above properties. Note that the attention output in Eq. (1), using the definition of softmax, is equivalent to the following expression:

$$\mathrm{Attn}(\boldsymbol{q}_n, \boldsymbol{K}_n, \boldsymbol{V}_n) = \frac{\exp(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)^\top \cdot \boldsymbol{V}_n}{\sum_{i \in [n]} \exp(\langle \boldsymbol{k}_i, \boldsymbol{q}_n \rangle)}.$$

Thus, to compute the attention output we need to calculate:

1. The matrix-vector product between $\boldsymbol{V}_n$ and $\exp(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)$.
2. The partition function $\sum_{i \in [n]} \exp(\langle \boldsymbol{k}_i, \boldsymbol{q}_n \rangle)$.

Thus, our DS needs to efficiently approximate each of these two operations. The matrix-vector product $\exp(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)^\top \cdot \boldsymbol{V}_n$ can be approximated efficiently using standard sampling-based techniques.

Specifically, we make use of the row norm sampling approach [7, 6]. When multiplying two matrices $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{B} \in \mathbb{R}^{n \times p}$, we randomly sample an i.i.d. index $i \in [n]$ with probability proportional to the ℓ_2 norm of the i-th row in $\boldsymbol{B}$. Then, we estimate $\boldsymbol{A} \cdot \boldsymbol{B}$ by the average of the product between i-th column in $\boldsymbol{A}$ and i-th row in $\boldsymbol{B}$. With this approximation, we need only $O(\varepsilon^{-2} d \log n)$ samples to guarantee an ε multiplicative error in spectral norm for $\exp(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)^\top \cdot \boldsymbol{V}_n$. Luckily, it can be implemented in a streaming setting through a variant of reservoir sampling [25].

The more challenging task is the sublinear-time approximation of the partition function $\sum_{i \in [n]} \exp(\langle \boldsymbol{k}_i, \boldsymbol{q}_n \rangle)$. We construct a DS for computing this under the assumption that the keys in the token stream are organized into a sublinear ($o(n)$) number of clusters. To be more precise, we introduce the following notion of clusterability:

Definition 1 (Clusterability)**.** *For a positive integer m and a real-valued $\delta > 0$, a dataset of points $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots \boldsymbol{x}_n \in \mathbb{R}^d$ is considered (m, δ)-clusterable if there exists a size-m partition $\mathcal{C}_1, \mathcal{C}_2, \ldots \mathcal{C}_m \subseteq \{\boldsymbol{x}_i\}_{i=1}^n$ of the dataset satisfying the following conditions:*

- *$\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$ for every $i \neq j$ and $\bigcup_{j=1}^m \mathcal{C}_j = \{\boldsymbol{x}_i\}_{i=1}^n$.*
- *for every $j \in [m]$ and every distinct pair $\boldsymbol{y}, \boldsymbol{z} \in \mathcal{C}_j$, $\|\boldsymbol{y} - \boldsymbol{z}\|_2 \leq \delta$.*

We demonstrate that under the assumption that the stream of keys $\boldsymbol{k}_1, \boldsymbol{k}_2, \ldots \boldsymbol{k}_n$ is (m, δ)-clusterable as defined in Definition 1, with the number of clusters scaling sublinearly in stream length ($m = o(n)$), it is possible to construct a DS with sublinear memory space. The procedure for this DS is presented in Algorithm 1 which we refer to as SubGen.

To verify this in the practical settings, we plot key embeddings from open-source LLMs in Section 3.1 and observe that they are indeed well clusterable on their embedding space. This motivates us to utilize an efficient stream clustering algorithm on key embeddings. In the remainder of this section, we provide a detailed explanation for the execution of the algorithm while simultaneously analyzing it through a series of lemmas.

2.2 Matrix Product Data Structure

Here, we focus on the UpdateMatrixProduct primitive and establish its correctness by introducing invariants that are maintained throughout the stream processing. This primitive maintains and updates a list of s elements denoted by $\mathcal{M}$ in SubGen (Algorithm 1). Initially, this list is filled with `null` values. After processing the first token tuple $(\boldsymbol{q}_1, \boldsymbol{k}_1, \boldsymbol{v}_1)$, this list is populated with s copies of the first key and value $(\boldsymbol{k}_1, \boldsymbol{v}_1)$. The procedure UpdateMatrixProduct performs a variant of reservoir sampling upon observing any new token in the stream. At any iteration n of the stream, $\mathcal{M}$ is ensured to contain s i.i.d. samples chosen at random from $(\boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{k}_2, \boldsymbol{v}_2), \ldots (\boldsymbol{k}_n, \boldsymbol{v}_n)$ with probabilities proportional to $\|\boldsymbol{k}_i\|_2^2$. More precisely, the following invariants hold:

Lemma 1 (Correctness of UpdateMatrixProduct)**.** *For any positive integer s, at any iteration n of the stream in 1 the following properties are maintained:*

- $\mu = \sum_{i \in [n]} \|\boldsymbol{v}_i\|_2^2$.

Algorithm 1: SubGen: Sublinear Streaming Attention

1: **inputs:** stream of tokens $(\boldsymbol{q}_n, \boldsymbol{k}_n, \boldsymbol{v}_n)$ for $n \in \mathbb{N}$, parameter $\delta > 0$, positive integers s, t
2: initialize $\mu \leftarrow 0$, $\mathcal{D} \leftarrow \emptyset$, $\mathcal{M} \leftarrow \left[\texttt{null}, \overset{\times s}{\cdots\cdots}\right]$
3: **repeat**
4: $\quad \mathcal{D} \leftarrow$ UpdateSoftmaxNormalizer$(\mathcal{D}, \delta, t, \boldsymbol{k}_n)$
5: $\quad \mathcal{M} \leftarrow$ UpdateMatrixProduct$(\mathcal{M}, s, \mu, \boldsymbol{k}_n, \boldsymbol{v}_n)$
6: $\quad \mu \leftarrow \mu + \|\boldsymbol{v}_n\|_2^2$
7: $\quad \boldsymbol{z}_n \leftarrow$ QueryStreamAttn$(\mathcal{D}, \mathcal{M}, s, t, \mu, \boldsymbol{q}_n)$
8: $\quad n \leftarrow n + 1$
9: $\quad$ **output** $\boldsymbol{z}_n$
10: **until** Token stream ends

Procedure UpdateSoftmaxNormalizer $(\mathcal{D}, \delta, t, \boldsymbol{k})$
11: Suppose input set $\mathcal{D} = \{(\boldsymbol{x}_i, \mathcal{S}_i, n_i) : i \in [m]\}$
12: $i^* \leftarrow \arg\min_{i \in [m]} \|\boldsymbol{x}_i - \boldsymbol{k}\|_2$
13: **if** $\|\boldsymbol{k} - \boldsymbol{x}_{i^*}\|_2 \leq \delta$ **then**
14: $\quad n_{i^*} \leftarrow n_{i^*} + 1$
15: $\quad$ Suppose $\mathcal{S}_{i^*}$ is a list of t vectors in $\mathbb{R}^d$
16: $\quad$ **for** $j \in [t]$ **do**
17: $\quad\quad$ Flip a coin and with probability $p = \frac{1}{n_{i^*}}$, update the jth entry of $\mathcal{S}_{i^*}$ as $\mathcal{S}_{i^*}(j) \leftarrow \boldsymbol{k}$
18: $\quad$ **end for**
19: **else**
20: $\quad \mathcal{S}' \leftarrow \left[\boldsymbol{k}, \overset{\times t}{\cdots\cdots}\right]$ (contains t copies of $\boldsymbol{k}$)
21: $\quad \mathcal{D} = \mathcal{D} \cup \{(\boldsymbol{k}, \mathcal{S}', 1)\}$
22: **end if**
23: **return** $\mathcal{D}$

Procedure UpdateMatrixProduct $(\mathcal{M}, s, \mu, \boldsymbol{k}, \boldsymbol{v})$
24: Suppose $\mathcal{M}$ is a list of s tuples of vectors in $\mathbb{R}^d$
25: **for** $i \in [s]$ **do**
26: $\quad$ Flip a coin and with probability $p = \frac{\|\boldsymbol{v}\|_2^2}{\mu + \|\boldsymbol{v}\|_2^2}$, update the ith entry of $\mathcal{M}$ as $\mathcal{M}(i) \leftarrow (\boldsymbol{k}, \boldsymbol{v})$
27: **end for**
28: **return** $\mathcal{M}$

Procedure QueryStreamAttn $(\mathcal{D}, \mathcal{M}, s, t, \mu, \boldsymbol{q})$
29: $\boldsymbol{z} \leftarrow \sum_{(\boldsymbol{k}, \boldsymbol{v}) \in \mathcal{M}} \frac{\mu}{s \cdot \|\boldsymbol{v}\|_2^2} \cdot \exp(\langle \boldsymbol{q}, \boldsymbol{k} \rangle) \cdot \boldsymbol{v}$
30: $\tau \leftarrow \sum_{(\boldsymbol{x}, \mathcal{S}, n') \in \mathcal{D}} \frac{n'}{t} \cdot \sum_{\boldsymbol{k} \in \mathcal{S}} \exp(\langle \boldsymbol{q}, \boldsymbol{k} \rangle)$
31: **return** $\boldsymbol{z}/\tau$

- *$\mathcal{M}$ is a list of s i.i.d. samples from $\{(\boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{k}_2, \boldsymbol{v}_2), \ldots (\boldsymbol{k}_n, \boldsymbol{v}_n)\}$ where the probability distribution for each element $j \in [s]$ is $\Pr[\mathcal{M}(j) = (\boldsymbol{k}_i, \boldsymbol{v}_i)] = \frac{\|\boldsymbol{v}_i\|_2^2}{\sum_{l \in [n]} \|\boldsymbol{v}_l\|_2^2}$ for $i \in [n]$.*

Proof. The first property is trivial because μ is initialized at zero and is updated in line 6 of the algorithm by adding the squared norms of $\boldsymbol{v}_i$'s. The proof of the second invariance is by

induction. The base of induction holds for $n = 1$ because after processing the first token by procedure UpdateMatrixProduct we have $\Pr[\mathcal{M}(j) = (\boldsymbol{k}_1, \boldsymbol{v}_1)] = \frac{\|\boldsymbol{v}_1\|_2^2}{\|\boldsymbol{v}_1\|_2^2} = 1$ for $j \in [s]$.

Now suppose that the inductive hypothesis holds for n and we prove it must also hold for $n+1$. For any $j \in [s]$ in line 24 of Algorithm 1 with probability $p = \frac{\|\boldsymbol{v}_{n+1}\|_2^2}{\mu + \|\boldsymbol{v}_{n+1}\|_2^2}$, $\mathcal{M}(j)$ gets updated to $(\boldsymbol{k}_{n+1}, \boldsymbol{v}_{n+1})$. Since we showed that $\mu = \sum_{i \in [n]} \|\boldsymbol{v}_i\|_2^2$ we have:

$$\Pr[\mathcal{M}(j) = (\boldsymbol{k}_{n+1}, \boldsymbol{v}_{n+1})] = \frac{\|\boldsymbol{v}_{n+1}\|_2^2}{\sum_{l \in [n+1]} \|\boldsymbol{v}_l\|_2^2}.$$

Moreover with probability $1 - p = \frac{\mu}{\mu + \|\boldsymbol{v}_{n+1}\|_2^2}$, $\mathcal{M}(j)$ keeps its previous value. Using the inductive hypothesis we have that for every $i \in [n]$:

$$\Pr[\mathcal{M}(j) = (\boldsymbol{k}_i, \boldsymbol{v}_i)] = \frac{\|\boldsymbol{v}_i\|_2^2}{\sum_{l \in [n]} \|\boldsymbol{v}_l\|_2^2} \cdot \frac{\sum_{l \in [n]} \|\boldsymbol{v}_l\|_2^2}{\sum_{l \in [n+1]} \|\boldsymbol{v}_l\|_2^2} = \frac{\|\boldsymbol{v}_i\|_2^2}{\sum_{l \in [n+1]} \|\boldsymbol{v}_l\|_2^2}.$$

This completes the proof. □

2.3 Softmax Normalizer (Partition Function) DS

Here we delve into a detailed discussion of the UpdateSoftmaxNormalizer primitive. This primitive constructs and maintains a DS denoted by $\mathcal{D}$, enabling accurate approximation of the partition function in the softmax denominator for any query. A crucial requirement for the efficiency of this primitive is that the key tokens must be (m, δ)-clusterable, as per Definition 1. Our algorithm locates and stores a subsampled representation of each cluster in $\mathcal{D}$ in a small memory. Particularly, to achieve sublinear memory complexity, instead of keeping all keys in each cluster which would require $O(n)$ memory space, we maintain only a random subset of t samples from each cluster.

Initially, $\mathcal{D}$ is an empty set. As new tokens in the stream are processed, new clusters get added to this set. Each cluster is characterized by a representative point, which is the first key assigned to that cluster by our algorithm. Throughout stream processing, we compute the distance between the new key token and each existing cluster. Here the distance to an existing cluster is defined as the distance to the aforementioned representative of the cluster. If there is a cluster whose distance is less than δ, then the token is assigned to the nearest cluster, and we update our random samples of keys from this cluster using reservoir sampling. If the distance from all existing clusters is more than δ, we introduce a new cluster in $\mathcal{D}$, and the new key becomes the representative of this new cluster. At any point in the stream, this algorithm identifies at most m clusters if the keys so far are (m, δ)-clusterable. If m grows sublinearly in the stream length n, the memory and update time of our algorithm will be sublinear as well. Formally, we prove that the following invariant holds:

Lemma 2 (Correctness of UpdateSoftmaxNormalizer)**.** *For any $\delta > 0$, any positive integer t, at any iteration n of the stream in Algorithm 1 the following properties are maintained. $\mathcal{D}$ is a set of m items of the form $\mathcal{D} = \{(\boldsymbol{x}_i, \mathcal{S}_i, n_i) : i \in [m]\}$, where there exists a partition of keys into m disjoint subsets $\mathcal{C}_1, \mathcal{C}_2, \ldots \mathcal{C}_m \subseteq \{\boldsymbol{k}_i\}_{i=1}^n$ satisfying $\bigcup_{j=1}^m \mathcal{C}_j = \{\boldsymbol{k}_i\}_{i=1}^n$ and $\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$ for every $i \neq j$, such that for every $i \in [m]$:*

1. $\boldsymbol{x}_i \in \mathcal{C}_i$,

2. $n_i = |\mathcal{C}_i|$,

3. $\|\boldsymbol{x}_i - \boldsymbol{k}'\|_2 \leq \delta$ *for every* $\boldsymbol{k}' \in \mathcal{C}_i$,

4. $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 > \delta$ *for every* $i \neq j$,

5. $\mathcal{S}_i$ *is a set of* t *i.i.d. uniform samples from the set* $\mathcal{C}_i$.

Proof. The proof is by induction on the stream length n. The base of induction trivially holds for $n = 0$, where $\mathcal{D}$ is an empty set. To prove the inductive step suppose that the inductive hypothesis holds for some n. Specifically, suppose that $\mathcal{D}$ is a set of m items of the form $\mathcal{D} = \{(\boldsymbol{x}_i, \mathcal{S}_i, n_i) : i \in [m]\}$ and there exists a partition of keys into m disjoint subsets $\mathcal{C}_1, \mathcal{C}_2, \ldots \mathcal{C}_m \subseteq \{\boldsymbol{k}_i\}_{i=1}^n$ as per in the lemma statement, such that for every $i \in [m]$: **(1)** $\boldsymbol{x}_i \in \mathcal{C}_i$, **(2)** $n_i = |\mathcal{C}_i|$, **(3)** $\|\boldsymbol{x}_i - \boldsymbol{k}'\|_2 \leq \delta$ for every $\boldsymbol{k}' \in \mathcal{C}_i$, **(4)** $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 > \delta$ for every $i \neq j$, and **(5)** $\mathcal{S}_i$ is a set of t i.i.d. uniform samples from the set $\mathcal{C}_i$. Given this assumption, we prove that the inductive step also holds for after processing the $(n+1)$-th key in the stream $\boldsymbol{k}_{n+1}$.

In the next iteration, specifically in line 12 of UpdateSoftmaxNormalizer, the algorithm finds the index $i^* \in [m]$ such that $\|\boldsymbol{x}_{i^*} - \boldsymbol{k}_{n+1}\|_2$ is minimized. Two cases arise:

Case 1: $\|\boldsymbol{x}_{i^*} - \boldsymbol{k}_{n+1}\|_2 \leq \delta$**.** In this case, the algorithm increments $n_{i^*} \leftarrow n_{i^*} + 1$ in line 14. Consider the new partitioning of the keys defined as $\mathcal{C}'_i = \mathcal{C}_i$ for $i \neq i^*$ and $\mathcal{C}'_{i^*} = \mathcal{C}_{i^*} \cup \{\boldsymbol{k}_{n+1}\}$. It follows from the inductive hypothesis that for every $i \in [m]$: **(1)** $\boldsymbol{x}_i \in \mathcal{C}'_i$, **(2)** $n_i = |\mathcal{C}'_i|$, **(3)** $\|\boldsymbol{x}_i - \boldsymbol{k}'\|_2 \leq \delta$ for every $\boldsymbol{k}' \in \mathcal{C}'_i$, and **(4)** $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 > \delta$ for every $i \neq j$ hold after the $n+1$-th iteration. Furthermore, since the algorithm does not alter the lists $\mathcal{S}_i$ for $i \neq i^*$, we have that **(5)** $\mathcal{S}_i$ is a set of t i.i.d. uniform samples from the set $\mathcal{C}'_i$ for any $i \neq i^*$. On the other hand, the algorithm in line 17 performs reservoir sampling on the set $\mathcal{S}_{i^*}$ with new element $\boldsymbol{k}_{n+1}$ which implies that $\mathcal{S}_{i^*}$ is a set of t i.i.d. uniform samples from the set $\mathcal{C}'_{i^*}$. This completes the inductive step in the first case.

Case 2: $\|\boldsymbol{x}_{i^*} - \boldsymbol{k}_{n+1}\|_2 > \delta$**.** In this case, the algorithm adds a new element to $\mathcal{D}$, thus, the updated set is $\mathcal{D}' = \{(\boldsymbol{x}_i, \mathcal{S}_i, n_i) : i \in [m+1]\}$ with $\boldsymbol{x}_{m+1} = \boldsymbol{k}_{n+1}$ and $n_{m+1} = 1$. If we consider the new partitioning of keys to be $\mathcal{C}_1, \mathcal{C}_2, \ldots \mathcal{C}_m, \mathcal{C}_{m+1}$, where $\mathcal{C}_{m+1} = \{\boldsymbol{k}_{n+1}\}$, we can use the inductive hypothesis to deduce that for any $i \in [m+1]$: **(1)** $\boldsymbol{x}_i \in \mathcal{C}_i$, **(2)** $n_i = |\mathcal{C}_i|$, **(3)** $\|\boldsymbol{x}_i - \boldsymbol{k}'\|_2 \leq \delta$ for every $\boldsymbol{k}' \in \mathcal{C}_i$, and **(4)** $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 > \delta$ for every $i \neq j$ hold after the $n+1$-th iteration of the stream. Furthermore, $\mathcal{S}_{m+1}$ is defined to be a list of t copies of $\boldsymbol{k}_{n+1}$, thus, **(5)** $\mathcal{S}_i$ is a set of t i.i.d. uniform samples from the set $\mathcal{C}_i$ for any $i \in [m+1]$. This completes the inductive step in this case and also concludes the proof of the lemma. □

2.4 Streaming Attention: Main Theorem

Now we are ready to analyze the end-to-end performance of SubGen and prove the main theorem. We show that, given the data structures created throughout the stream and analyzed in Lemma 1

and Lemma 2, the primitive QueryStreamAttn can efficiently output an accurate approximation to the streaming attention, satisfying Eq. (1).

Our analysis unfolds in two steps. First, we establish that the data structures created by UpdateSoftmaxNormalizer and UpdateMatrixProduct can be stored in small memory and updated very quickly if the sequence of keys is clusterable into a sublinear number of clusters. Then we show that the QueryStreamAttn can use these data structures to produce an accurate attention output for any given query. Our main result is as follows:

Theorem 1 (Efficiency and Correctness of Algorithm 1)**.** *For any $\delta, r, \varepsilon > 0$, any positive integers n, d, and any sequence of tokens $(\boldsymbol{q}_1, \boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{q}_2, \boldsymbol{k}_2, \boldsymbol{v}_2), \ldots (\boldsymbol{q}_n, \boldsymbol{k}_n, \boldsymbol{v}_n)$ where $\boldsymbol{q}_i, \boldsymbol{k}_i, \boldsymbol{v}_i \in \mathbb{R}^d$, suppose that the followings hold*

- $t = \Omega\left(\varepsilon^{-2} \cdot e^{2\delta \cdot r} \log n\right)$,
- $s = \Omega(\varepsilon^{-2} \cdot d)$,
- $\|\boldsymbol{q}_n\|_2 \leq r$.

Then, SubGen *(Algorithm 1) at n-th step of the stream processing outputs a vector $\boldsymbol{z}_n \in \mathbb{R}^d$ that satisfies Eq. (1) with probability at least 0.99. Furthermore, if the keys $\boldsymbol{k}_1, \boldsymbol{k}_2, \ldots \boldsymbol{k}_n$ are (m, δ)-clusterable as per Definition 1, then both the total memory of the algorithm and its runtime during the n-th iteration is bounded by $O(d \cdot (mt + s))$.*

Proof. We start the correctness proof by observing that all preconditions of Lemma 2 are satisfied, allowing us to invoke this lemma. Let the partition of keys into disjoint subsets be denoted by $\mathcal{C}_1, \mathcal{C}_2, \ldots \mathcal{C}_{m'} \subseteq \{\boldsymbol{k}_i\}_{i=1}^n$ satisfying $\bigcup_{j=1}^{m'} \mathcal{C}_j = \{\boldsymbol{k}_i\}_{i=1}^n$ and $\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$ for every $i \neq j$ as per Lemma 2 for some positive integer m'. Rewriting the partition function in the attention denominator gives:

$$\sum_{j \in [n]} \exp(\langle \boldsymbol{k}_j, \boldsymbol{q}_n \rangle) = \sum_{i \in [m']} \sum_{\boldsymbol{k}' \in \mathcal{C}_i} \exp(\langle \boldsymbol{k}', \boldsymbol{q}_n \rangle).$$

Now by property **(3)** in Lemma 2 and triangle inequality, for every $i \in [m']$ and every $\boldsymbol{k}', \boldsymbol{k}'' \in \mathcal{C}_i$ we have:

$$\left\|\boldsymbol{k}' - \boldsymbol{k}''\right\|_2 \leq \left\|\boldsymbol{k}' - \boldsymbol{x}_i\right\|_2 + \left\|\boldsymbol{k}'' - \boldsymbol{x}_i\right\|_2 \leq 2\delta.$$

Therefore, using the precondition of the theorem on $\|\boldsymbol{q}_n\|_2 \leq r$ we have

$$\exp(\langle \boldsymbol{k}', \boldsymbol{q}_n \rangle) / \exp(\langle \boldsymbol{k}'', \boldsymbol{q}_n \rangle) \leq e^{2\delta \cdot r}.$$

Using the above inequality and the assumption in the theorem statement regarding $t = \Omega\left(\varepsilon^{-2} \cdot e^{2\delta \cdot r} \log n\right)$ combined with the properties **(2)** and **(5)** proved in Lemma 2, we can invoke Chernoff-Hoeffding inequality (see e.g., [16]) along with union bound to conclude that the following holds simultaneously for all $i \in [m']$ with probability at least $1 - \frac{1}{\mathrm{poly}(n)}$:

$$\frac{n_i}{t} \cdot \sum_{\boldsymbol{k}' \in \mathcal{S}_i} \exp(\langle \boldsymbol{q}_n, \boldsymbol{k}' \rangle) \in (1 \pm \varepsilon/3) \cdot \sum_{\boldsymbol{k}' \in \mathcal{C}_i} \exp(\langle \boldsymbol{k}', \boldsymbol{q}_n \rangle)$$

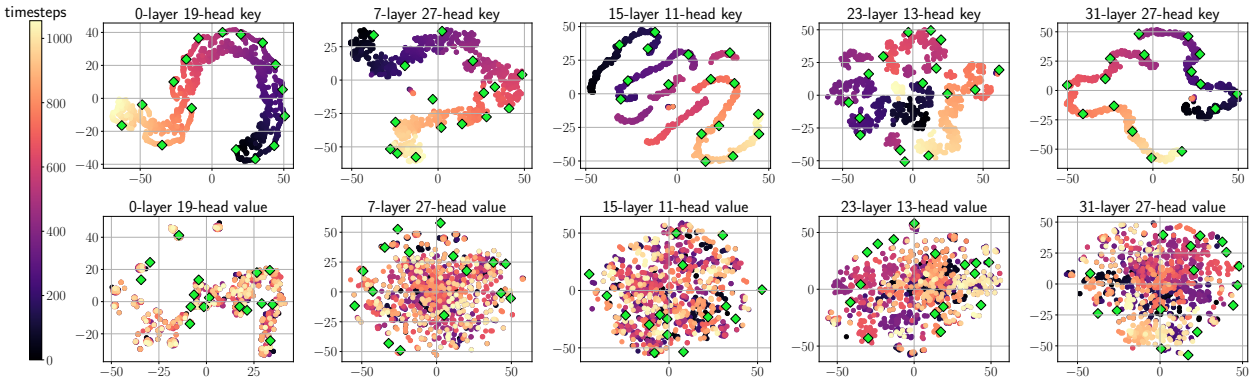

Figure 1: A t-SNE plot of cached keys (first row) and values (second row) embeddings over 1024 timesteps from Llama2-7B using MT Bench dataset. We pick ℓ-layer where $\ell \in \{0, 7, 15, 23, 31\}$ and head IDs are chosen uniformly at random. Key embeddings are more clusterable than value ones. The green dots represent the centers from the greedy k-center algorithm [8] where k=16.

Since the terms above are positive, by summing up the given inequality for all $i \in [m']$, we find that the quantity τ computed in line 27 of Algorithm 1 satisfies the following:

$$\Pr\left[\tau \in (1 \pm \varepsilon/3) \sum_{j \in [n]} \exp(\langle \boldsymbol{k}_j, \boldsymbol{q}_n \rangle)\right] \geq 0.995 \tag{5}$$

Next, we invoke Lemma 1 to derive an error bound on the approximate matrix-vector product between the softmax vector and the matrix of values $\boldsymbol{V}_n$. By leveraging well-established techniques in approximate matrix products, such as the standard result from [7], and using the conclusion of Lemma 1 regarding $\mathcal{M}$ as a list of $s = \Omega(\varepsilon^{-2} \cdot d)$ i.i.d. sample from the probability distribution $\Pr[\mathcal{M}(j) = (\boldsymbol{k}_i, \boldsymbol{v}_i)] = \frac{\|\boldsymbol{v}_i\|_2^2}{\sum_{l \in [n]} \|\boldsymbol{v}_l\|_2^2}$ for $i \in [n]$ for $i \in [n]$ and $j \in [s]$, we have that vector $\boldsymbol{z}$ computed in line 26 of Algorithm 1 satisfies the following inequality with a probability of at least 0.995:

$$\left\| \boldsymbol{z} - \exp(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)^\top \cdot \boldsymbol{V}_n \right\|_2 \leq \frac{\varepsilon}{3} \|\exp(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)\|_2 \|\boldsymbol{V}_n\|_{op} \tag{6}$$

Now by combining inequalities in Eq. (5) and Eq. (6) using union bound and triangle inequality we find that the output of Algorithm 1 computed in line 28 as $\boldsymbol{z}/\tau$ satisfies the following with probability at least 0.99

$$\left\| \boldsymbol{z}/\tau - \texttt{softmax}(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)^\top \cdot \boldsymbol{V}_n \right\|_2 \leq \varepsilon \|\texttt{softmax}(\boldsymbol{K}_n \cdot \boldsymbol{q}_n)\|_2 \|\boldsymbol{V}_n\|_{op}.$$

This completes the correctness proof.

Memory and Runtime. First, note that the memory requirement for storing the list $\mathcal{M}$ in Algorithm 1 is $O(sd)$ because it contains s pairs of d-dimensional vectors. Next, to bound the memory requirement for storing $\mathcal{D}$ we need to bound the size of this set which we denoted by m'. According to properties **(1)** and **(4)** in Lemma 2, for every $i \in [m']$ there exist $\boldsymbol{x}_i \in \{\boldsymbol{k}_1, \boldsymbol{k}_2, \ldots \boldsymbol{k}_n\}$

	$n = 5$k		$n = 7$k		$n = 9$k	
Algorithm	Cache Size (GB)	Accuracy	Cache Size (GB)	Accuracy	Cache Size (GB)	Accuracy
Exact	2.351	0.98	3.488	1.0	4.613	0.68
Sink [26]	1.511 (35% ↓)	0.56	2.012 (42% ↓)	0.56	2.262 (50% ↓)	0.38
H2O [28]	1.511 (35% ↓)	0.66	2.012 (42% ↓)	0.58	2.262 (50% ↓)	0.38
SubGen (this work)	1.512 (35% ↓)	**0.86**	2.012 (42% ↓)	**0.66**	2.262 (50% ↓)	**0.44**

Table 1: Results on accuracy of line retrieval from LongEval [13] dataset with context length 5k-9k. Under the sublinear budgets on cache size, the proposed algorithm based on k-center algorithm outperforms other methods over all sequence lengths.

such that $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 > \delta$ for $i \neq j$. Given the assumption in the theorem statement that keys are (m, δ)-clusterable, by the definition of clusterability in Definition 1 along with the pigeonhole principle, we must have $m' \leq m$. Therefore storing $\mathcal{D}$ will require $O(m'td) = O(mtd)$ because it is a set of m' elements, and each element of this set is a list of t vectors in dimension d.

Three major operations dominate the runtime of the n-th iteration. Firstly, executing UpdateSoftmaxNormalizer requires computing m' distances in line 12 that takes $O(md)$ time. Additionally, the for loop in line 16 takes $O(td)$ time. Secondly, UpdateMatrixProduct has a runtime bounded by $O(sd)$. Thirdly, running QueryStreamAttn involves $O(sd)$ operations in line 26 and $O(m'td) = O(mtd)$ operations in line 27. As a result, the total runtime of Algorithm 1 in n-th iteration is $O(mtd + sd)$. □

Theorem 1 demonstrates that if the keys can be clustered into some sublinear number $m = n^{1-\Omega(1)}$ of clusters with diameters at most δ, and the queries have bounded ℓ_2 norms of at most r such that the product of the cluster diameter and maximum ℓ_2 norm of queries is bounded by $\delta r = o(\log n)$, then Algorithm 1 operates with sublinear $O\left(\varepsilon^{-2} \cdot mdn^{o(1)}\right) = O\left(\varepsilon^{-2} \cdot dn^{1-\Omega(1)}\right)$ memory and runtime. We summarize this in the following corollary:

Corollary 1. *Suppose the preconditions of Theorem 1 hold. If the diameter of key token clusters δ and the maximum ℓ_2 norm of queries r satisfy $\delta r = o(\log n)$, then the total memory and runtime of Theorem 1 are bounded by $O\left(\varepsilon^{-2} \cdot dmn^{o(1)}\right)$. Moreover, if the number of key token clusters m grows as a sublinear function of n, i.e., as $m = n^{1-\Omega(1)}$, then the memory and runtime are bounded by $O\left(\varepsilon^{-2} \cdot dn^{1-\Omega(1)}\right)$.*

3 Experiments

In this section, we report the empirical results of the proposed algorithm with memory footprint reduction and performance on question-answering benchmark datasets. For all experiments, we use a single NVIDIA RTX6000 GPU with 48 GB VRAM.

3.1 Ablation Study on Clusterability

We first demonstrate that cached embeddings over long token generations are indeed well clusterable. To this end, we collect key and value embeddings from Llama-2-7B [23] with MT Bench dataset [29] while the model generates a sequence of 1024 tokens. We then visualize the embeddings using t-SNE [24] across various layers and heads, identifying cluster center points through the greedy k-center algorithm [8].

As illustrated in Fig. 1, our observations reveal that key embeddings (first row) exhibit a higher degree of clusterability compared to value embeddings across all randomly selected layers and heads. Furthermore, we note that the cluster centers (indicated by green dots) corresponding to the key embeddings are evenly distributed across the entire embedding space. In particular, the key embeddings demonstrate significant dispersion across different time steps, and their cluster centers are distributed over the entire embedding space.

This behavior comes from the use of Rotary Position Embedding (RoPE) [21] in Llama-2-type models which introduces rotational transformations to both query and key embeddings based on their relative positions. Hence, the key embeddings appear to be well-separated in their projected space, while the values show an unstructured and random distribution within their space. These findings serve as a motivation for the development of an efficient key-value (KV) compression algorithm that leverages the clustering properties of key embeddings.

3.2 End-to-end Text Generation

We next evaluate our proposed algorithm on long-context line retrieval task in LongEval [13][2] benchmark. The task involves long-context line retrieval from extensive documents, each comprising multiple lines, complete with line numbers and topics. The objective is to precisely retrieve a specified number of lines corresponding to a target topic. We vary the number of lines, representing the number of targets, to 200, 300, and 400 and they correspond to sequence lengths of n =5,000, 7,000, and 9,000, respectively. Each dataset contains 50 distinct questions, and we systematically extract the number from the generated answers and compute accuracies. The answers are generated employing the longchat-7B model[3], which is a fine-tuned version of the Llama-2-7B model with long-range context length.

We compare our method to two KV cache compression algorithms; H2O [28], which retains cached tokens with high cumulative attention scores, and Attention Sink [26], a method that deterministically selects some initial and recent tokens. Specifically, both of these prior works have highlighted the significance of recent token embeddings in generating meaningful responses. To leverage this insight, we integrate it with our clustering approach. More precisely, our strategy consistently retains the most recent r token embeddings, in addition to k centers selected from the remaining tokens. In a streaming context, this strategy is often referred to as a *sliding window*. We apply the greedy k-center clustering algorithm once to compress the entire KV caches. To make comparisons fair, we set cache memory budgets of all algorithms identical (i.e., $r + k$), which scales sublinearly with the context length denoted as n.

[2] https://github.com/DachengLi1/LongChat/blob/longeval
[3] https://huggingface.co/lmsys/longchat-7b-v1.5-32k

The results are reported in Table 1. We observe that our clustering-based method consistently outperforms other algorithms across all sequence lengths. For instance, we achieve an accuracy of 44% while utilizing only half of the cached KV embeddgins with a length of 9k tokens, whereas both H2O and Sink can achieve accuracies 10% lower. This finding suggests that maintaining the embedding information holds greater significance in sustaining the performance of LLMs compared to attention scores and positional information.

4 Conclusion

In this work, we develop SubGen, an efficient KV cache compression algorithm via stream clustering. Our motivation is that cached keys are well clusterable in their embedding space and we apply a greedy-type clustering algorithm to find the most representative embeddings. Under assumptions on bounded query norm and clusterability, we analyze that our algorithm can guarantee a spectral error bound with sublinear time and memory. We further integrate keeping recent tokens to the proposed clustering approach. For zero-shot line retrieval tasks, our algorithm outperforms other KV cache compression algorithms with the same memory budget.

References

[1] Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Alman, J. and Song, Z. Fast attention requires bounded entries. *arXiv preprint arXiv:2302.13214*, 2023.

[3] Charikar, M., Chekuri, C., Feder, T., and Motwani, R. Incremental clustering and dynamic information retrieval. In *Symposium on the Theory of Computing (STOC)*, 1997.

[4] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. 2023. URL `https://vicuna.lmsys.org`.

[6] Cohen, M. B., Nelson, J., and Woodruff, D. P. Optimal Approximate Matrix Product in Terms of Stable Rank. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, 2016.

[7] Drineas, P. and Kannan, R. Fast Monte-Carlo algorithms for approximate matrix multiplication. In *Foundations of Computer Science (FOCS)*, 2001.

[8] Dyer, M. E. and Frieze, A. M. A simple heuristic for the p-centre problem. *Operations Research Letters*, 1985.

[9] Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. *arXiv preprint arXiv:2310.01801*, 2023.

[10] Han, I., Jayaram, R., Karbasi, A., Mirrokni, V., Woodruff, D. P., and Zandieh, A. Hyper-attention: Long-context attention in near-linear time. *International Conference on Learning Representations (ICLR)*, 2024.

[11] Ho, J., Chan, W., Saharia, C., Whang, J., Gao, R., Gritsenko, A., Kingma, D. P., Poole, B., Norouzi, M., Fleet, D. J., et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022.

[12] Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., Chen, K., Dong, H., and Wang, Y. Flashdecoding++: Faster large language model inference on gpus. *arXiv preprint arXiv:2311.01282*, 2023.

[13] Li, D., Shao, R., Xie, A., Sheng, Y., Zheng, L., Gonzalez, J., Stoica, I., Ma, X., and Zhang, H. How Long Can Context Length of Open-Source LLMs truly Promise? In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, 2023.

[14] Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., and Shrivastava, A. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *Neural Information Processing Systems (NeurIPS)*, 2023.

[15] Liu, Z., Wang, J., Dao, T., Zhou, T., Yuan, B., Song, Z., Shrivastava, A., Zhang, C., Tian, Y., Re, C., et al. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning (ICML)*, 2023.

[16] McDiarmid, C. Concentration. In *Probabilistic methods for algorithmic discrete mathematics*. Springer, 1998.

[17] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[18] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[19] Sarlos, T., Song, X., Woodruff, D., and Zhang, Q. Hardness of low rank approximation of entrywise transformed matrix products. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[20] Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., and Zhang, C. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning (ICML)*, 2023.

[21] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[22] Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T. B. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

[23] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[24] Van der Maaten, L. and Hinton, G. Visualizing data using t-SNE. *Journal of Machine Learning Research (JMLR)*, 2008.

[25] Vitter, J. S. Random sampling with a reservoir. *ACM Transactions on Mathematical Software (TOMS)*, 1985.

[26] Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

[27] Zandieh, A., Han, I., Daliri, M., and Karbasi, A. Kdeformer: Accelerating transformers via kernel density estimation. In *International Conference on Machine Learning (ICML)*, 2023.

[28] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *Neural Information Processing Systems (NeurIPS)*, 2023.

[29] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023. URL https://github.com/lm-sys/FastChat/tree/main/fastchat/llm_judge.